Module 12 - Benchmarking & Integration

Hardware Concurrency, Memory Models & Lock-Free Design | SPSC ring buffer (Module 11) evaluated with JMH | run date 2026-10-10

Item	Value
Environment	i7-13700HX (16 cores / 24 logical), 16 GB, Windows 11, Oracle JDK 27+35, Maven 3.10.0, JMH 1.37, no extra JVM flags, threads not pinned
Power / thermal	AC connected; power plan named "Acer" (not High performance); no noticeable heating, but no thermal measurement taken (throttling not ruled out)
Tests	59 run, 0 failures, 0 errors (mvn clean test)
JMH settings	5 x 1 s warm-up, 10 x 1 s measurement, 3 forks; total 4m34s (throughput) + 2m17s (latency)
Evidence label	TEAM-MEASURED (single machine, single JDK)

1. What was built

SpscThroughputBenchmark: plain vs padded SPSC (capacity 64 and 1024) plus ArrayBlockingQueue (ABQ) as a reference, 1 producer + 1 consumer per group; 1 op = 1 completed transfer, consumer row reported. **SpscLatencyBenchmark**: ping-pong round trip (depth 1), SampleTime mode. **OffsetProbe** (Unsafe) and **JolLayout** (JOL 0.17) for field layout. **run_module12.ps1** and **analyze_m12.py** automate environment capture, tests, JMH runs, CSVs and charts. The Module 11 benchmark skeleton was left unchanged: static review found a 2x2x2 @Param cross-product, spin loops without Control.stopMeasurement (can hang at iteration end), ABQ failed attempts counted as operations, and a group aggregate that double-counts transfers.

2. Memory-layout validation (JDK 27; Unsafe and JOL agree)

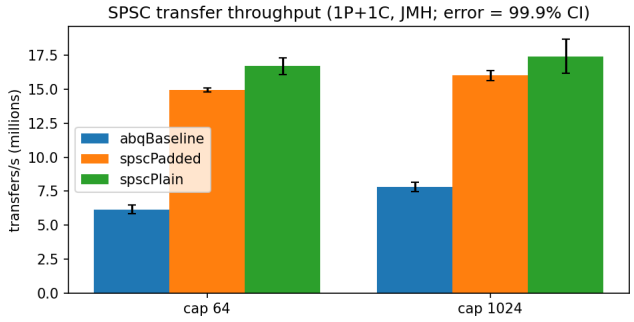
Class	producerSeq	consumerSeq	mask / entries	Reading
SpscRingBuffer	8	16	24 / 28	all hot fields within 32 B: same line likely
PaddedSpscRingBuffer	64	184	248 / 252	sequences 120 B apart; each >= 64 B from mask/entries

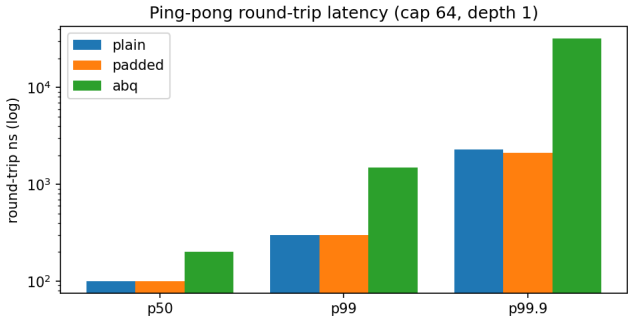
The JVM preserved the padding on this JDK, so the two sequences cannot share a 64 B line. A different JDK (21.0.10 in the authoring sandbox) produced different offsets, so layout must be re-verified per JVM. **NOT PROVEN:** that false sharing occurs or is removed - no hardware counters were available and the 64 B line size was assumed, not queried.

3. Results (99.9% confidence interval from JMH)

Throughput (transfers/s)	Capacity 64	Capacity 1024
SpscRingBuffer (plain)	16.71 M +/- 0.61 M	17.43 M +/- 1.26 M
PaddedSpscRingBuffer	14.95 M +/- 0.17 M	16.02 M +/- 0.36 M
ArrayBlockingQueue (reference)	6.16 M +/- 0.32 M	7.82 M +/- 0.35 M

Round trip, ns (cap 64, depth 1)	mean	p50	p99	p99.9	max
plain	167	100	300	2,300	1.6 ms
padded	167	100	300	2,100	3.3 ms
ABQ	459	200	1,500	32,096	6.6 ms





4. Interpretation - what the data supports

- Both SPSC queues were about 2.1-2.7x faster than ABQ. This reflects lock-free hand-off versus a locked multi-producer/consumer queue with different semantics; it is a reference point, not a like-for-like comparison.

- **Padding did not help.** At capacity 64 plain beat padded by about 12% with non-overlapping intervals. At 1024 the intervals overlap (plain fork means 15.6-19.4 M), so no difference is established. The padding hypothesis is not supported on this machine and JVM; the cause is NOT PROVEN without counters.

- Mean round trip is identical for plain and padded (about 167 ns). p50 = 100 ns sits at the Windows timer resolution, so rely on the mean and the tail, not p50 or a derived one-way figure. Millisecond maxima are OS scheduling, not queue behaviour.

- Fork-to-fork spread for plain is about 10% (unpinned threads on a 24-thread hybrid CPU); treat smaller differences as noise.

5. Integration status and limits

Item	Status
Module 11 correctness (59 tests, 5M-event stress)	TEAM-MEASURED; 59 tests re-run in Module 12, 0 failures
Throughput, latency, field offsets (JDK 27)	TEAM-MEASURED (single machine, unpinned)
False sharing present / removed by padding	NOT PROVEN (needs hardware counters)
Real LMAX Disruptor benchmark	NOT PROVEN / out of scope (documentation only)
M03, M06, M07, M08 benchmark and outcome results	PENDING LOCAL EXECUTION (see evidence matrix)
Project contract / Module 12 API contract	UNVERIFIED (not supplied)

Next steps: repeat pinned to two performance cores with the High performance power plan; add hardware counters (perf c2c / perfnorm on Linux) before any false-sharing claim. **Reproduce:** run scripts\run_module12.ps1 from the project folder; outputs are in results/m12 (files under results/m12/quick are smoke runs, not evidence).